

THE GIFT OF FEEDBACK: IMPROVING ASR MODEL QUALITY BY LEARNING FROM USER CORRECTIONS THROUGH FEDERATED LEARNING

Lillian Zhou Yuxin Ding Mingqing Chen Harry Zhang
 Rohit Prabhavalkar Dhruv Guliani Giovanni Motta Rajiv Mathews

Google LLC, Mountain View, CA, U.S.A.
 {lqz, yxding, mingqing}@google.com

ABSTRACT

Automatic speech recognition (ASR) models are typically trained on large datasets of transcribed speech. As language evolves and new terms come into use, these models can become outdated and stale. In the context of models trained on the server but deployed on edge devices, errors may result from the mismatch between server training data and actual on-device usage. In this work, we seek to continually learn from on-device user corrections through Federated Learning (FL) to address this issue. We explore techniques to target fresh terms that the model has not previously encountered, learn long-tail words, and mitigate catastrophic forgetting. In experimental evaluations, we find that the proposed techniques improve model recognition of fresh terms, while preserving quality on the overall language distribution.

Index Terms— speech recognition, federated learning, deep learning, catastrophic forgetting, on-device training

1. INTRODUCTION

Human language is constantly shifting and evolving. In order to serve a high quality ASR model that can be deployed to user devices as an input mechanism, it is crucial to train on data that is representative of the actual, current vocabulary of user dictation. While it is possible to use proxy data for initial server-side training, stopping there means that the model will lag behind the ever-changing user distribution in terms of freshness and accuracy.

One way to improve model freshness is to make use of the natural feedback loop that occurs during usage. When the model makes recognition errors, users may make manual edits to the output text; in the ideal case, this points to a misrecognition that the model has made, and additionally gives us the corrected transcript to learn from. Paired with the original audio, these training examples are a treasure trove for improving model quality, containing up-to-date transcriptions that faithfully reflect actual user requirements. Federated Learning (FL) [1, 2] affords us a privacy-preserving mechanism to leverage this training data and these valuable user correction signals.

Related work has explored mining challenging training examples for model improvement [3, 4, 5, 6], but in this work we **utilize the model’s own errors, and user corrections thereof, to target fresh terms**. In this way, the model can improve on precisely the words it struggles with, and **learn fresh terms unseen in the training corpus snapshot on the server**.

However, not all user edits made naturally in the course of usage are necessarily true corrections to the original spoken utterance. In many cases, the edits may be revisions of original intent, and the resulting edited text may diverge entirely from the original audio. In this work, **we propose a simple method to target legitimate user corrections: filtering training examples to those that contain terms the model is likely to misrecognize**, for example fresh terms that did not exist when the server training data was collected, and would be unknown to the server model. These misrecognized words are likely to be the target of true user corrections. As shown in our experimental results, this approach successfully helps to target high-quality training examples, and fine-tuning on them improves model quality on these fresh terms.

This improvement on the targeted terms, however, can also come with the unintended pitfall of regression on the original distribution. **We compare a number of techniques for mitigating catastrophic forgetting [7, 8] in FL, including variants of weight averaging algorithms [9]**. We also reintroduce data from the original distribution, which in the FL context requires mixing centralized training in with federated rounds [10], to positive effect.

Finally, these fresh terms can be long-tail words, and training examples may be **exceedingly sparse**. We **propose two approaches to improve learning on the rarest examples**. We find that these combined techniques allow us to learn fresh terms without harming overall model quality.

2. METHODOLOGY

2.1. Federated Learning

Federated Learning [1, 2] is a technique to train a single centralized model in a distributed fashion. During each round of federated training, participating edge devices receive the

Word	Source	Seen Count
warnock	current events	85
webb	technology	48
addams	media	38
mbappe	sports	27
salman	current events	8
sumeru	media	8

Table 1. Examples of words used for filtering, based on fresh or trending terms at time of experimentation.

most recent centralized checkpoint, **locally train on available on-device training examples, and send only the model gradients back to the server, never any user data.** These updates are aggregated into the central model, which is then sent out again for the next federated round.

In the context of this work, on-device training examples consist of speech audio, the original transcript output by the inference model, along with the final text committed by the user after any edits.

2.2. Filtering Examples By Fresh Words

In order to target high-quality training examples, we generate a list of fresh words that are likely to be misrecognized by the model, and hence the target of legitimate user correction, rather than other edits. In the long term, this step should be automated, e.g., by aggregating over user corrections across devices through Federated Analytics [11]. Words that are corrected by a large number of users are likely to be true misrecognitions. **For this work, we experiment with a manually-curated list of 241 words selected from sources such as pop culture, current events, and recent technologies.** The terms vary greatly in how often they appear in on-device training examples (Fig 1), and some examples are shown in Table 1.

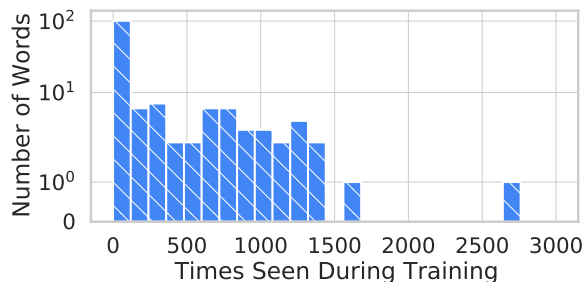


Fig. 1. Histogram of wordlist word frequency during training. Approximately 50% of words were seen after 300 rounds of training, with a wide range of frequency.

While training in FL, we filter the utterances available on-device to examples that contain at least one word from this wordlist and have a user edit. In this way, we target cases

where the user spoke one of these words, the model produced the wrong transcript, and the user corrected it.

Because these words are by nature long-tail and represent only a small portion of utterances, evaluating improvement is challenging. **To do so, we create a targeted testset focused on measuring quality on these rare words.** We compile 4-5 sentences containing each wordlist word: a combination of manually-generated sentences and sentences scraped from the web. **Then we generate corresponding audio using Text-to-Speech (TTS) [12], and use the resulting audio-sentence pairs as our targeted testset.** Due to the nature of TTS audio, and the fact that the terms used for the testset are inherently challenging and rare, including terms that were infrequently or never encountered during training, the resulting WER tends to be high, but serves as a useful benchmark to understand quality improvement on these targeted words. We further perform per-word error analysis before and after fine-tuning to directly understand how recognition of each term is affected.

2.3. Mitigating Forgetting

2.3.1. Static Checkpoint Averaging

To mitigate forgetting, we experiment with several techniques, starting with simple weight averaging [9]. Prior to evaluation, we average the weights of each federated checkpoint with the weights of the initial pre-trained checkpoint, modified by a scaling factor (Fig 2a). Federated rounds proceed as usual, and are not impacted by this averaging.

2.3.2. Dynamic Checkpoint Averaging

As above, this approach averages the weights of each federated checkpoint with the initial checkpoint (Fig 2b). However, here each averaged checkpoint is then used to initialize the subsequent federated round. In other words, the federated training process is modified to initialize from the new averaged checkpoint, rather than from the purely federated checkpoint from the round before.

2.3.3. Mixture of Centralized and Federated Training

Finally, we perform server-side training in parallel with federated training (Fig 2c), as described in [10]. While FL rounds learn the on-device data, we simultaneously perform centralized training on the same server datasets used to train the initial pre-trained checkpoint. These centralized model updates are included along with those of the federated clients during aggregation at the end of each round.

2.4. Learning Long-tail Words

Because the fresh and misrecognized words tend to be long-tail words, training examples are difficult to come by. All but non-existent in the server training corpus, they are rare even

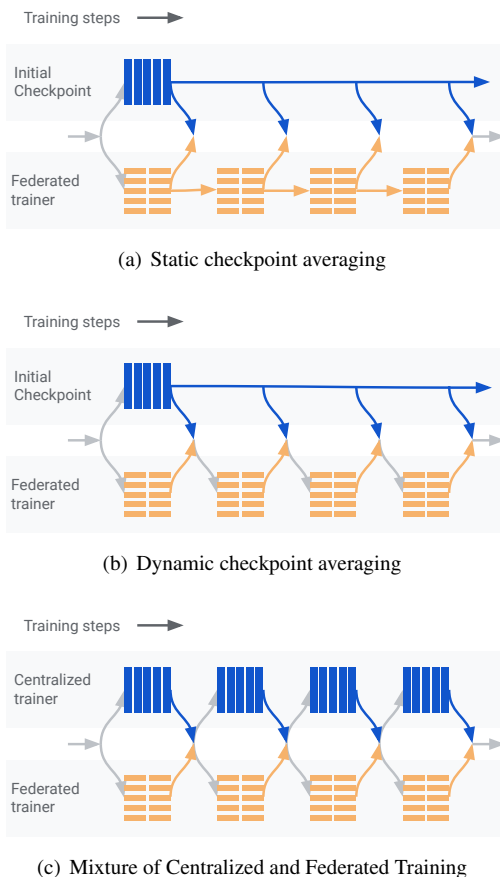


Fig. 2. Illustration of techniques to mitigate forgetting. Each shows consecutive federated training rounds (yellow), with varying types of server-side modification (blue), and progression of the centralized model being trained (gray).

1. 这些词本来就稀有
2. 这些稀有词内部，频率差别非常大

in on-device data. Within the long-tail words, availability also varies drastically from term to term; training on this imbalanced distribution results in little improvement on the least frequent words. To address this, we propose two techniques: probabilistic sampling and client loss weighting.

2.4.1. Probabilistic Sampling

概率采样：每个词有一个采样概率 p ，
词越常见， p 越小

Probabilistic sampling aims to artificially massage the training examples into a more uniform distribution by down-sampling the federated clients that only have examples of the most common words from the wordlist. Each word is assigned a sampling probability p , where $p \in (0, 1]$. The more frequent a word is in the original data distribution, the smaller its p value.

If the client data contains a targeted word w , then it is included in the FL round with probability p_w . If the client has multiple wordlist words, either within the same utterance or over multiple training examples, then its probability is the maximum among the words' sampling probabilities. Due to

the limited number of clients included in each round, this increases the occurrences of less frequent words seen during training.

每个词都有一个权重 w ，词越稀有， w 就越大，
每条语句的loss乘上它所含目标词里最大的 w ，再求和
这样认错稀有词的惩罚更重

2.4.2. Client Loss Weighting

Client loss weighting aims to more heavily penalize the wrong predictions of the rarest wordlist words. Each target word is assigned a client loss weight (w). The more frequent the word, the smaller the w . On the client, the loss is computed for each utterance, and then additionally scaled by the w of any wordlist word the utterance contains. If an utterance contains multiple target words, its w is the maximum among all the weights. The client loss is the sum of each utterance loss multiplied by the utterance loss weight:

$$\mathcal{L}^{\text{client}} = \sum_{u \in U} \max_{d \in D} (w_d) \cdot \text{loss}_u \quad (1)$$

where $\mathcal{L}^{\text{client}}$ is the client loss, computed over U utterances on the client and D words per utterance.

3. MODEL AND DATA

3.1. Architecture

All models described in this paper are end-to-end, streaming transducer models [13, 14, 15] based on the Conformer architecture [16]. Due to the cost of training in FL, we initially demonstrate the wordlist filtering approach in server-side simulation, before expanding to a production FL setup on user devices. For the simulation experiments, we use a streaming Conformer model [17], while in production, we use a variation of this model that utilizes cascading encoders [18].

One important consideration for on-device learning is computational efficiency. Research in deep learning has shown that neural networks benefit from being overparameterized [19, 20]. In particular, when seeking to learn new words in a deep ASR model, previous research has shown the majority of quality gains can be achieved by training only the topmost layers, such as the joint layer, or the joint and prediction network (which together we refer to as the decoder portion of the model) [5]. We adopt this setting; compared to training the entire model, this affords us significant memory savings. For example, for the production model [18], the decoder portion is only 40M parameters out of the entire 150M-parameter model. A static, frozen, and compressed encoder is used to generate encoder features, which are then used as input for decoder training.

3.1.1. On-Device Minimum Word Error Rate Training

In addition to standard RNN-t loss, we also incorporate Minimum Word Error Rate (MWER) loss [21], which is computed from the loss of each of the N -best hypotheses from beam

关键设计：只训练 decoder，冻结 encoder

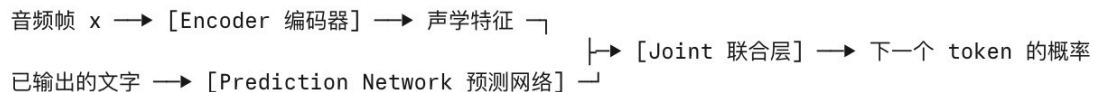
论文 [19, 20] 的观点是：神经网络通常过参数化，只调其中一小部分参数也能获得大部分收益。

论文 [5] 发现：让 ASR 模型学新词时，大部分提升来自训练最上面几层，也就是 joint 层，

或者 joint 加 prediction network。论文把这两部分合称为 decoder。

一、模型：流式 Conformer Transducer

这是 RNN-T 结构的端到端语音识别模型，由三个模块组成：



模块	作用	类比
Encoder	把音频转成声学特征，回答“听到了什么声音”	声学模型
Prediction Network	根据已经输出的文字预测接下来可能出现什么，回答“语言上接下来可能是什么”	语言模型
Joint	把声学特征和语言信息合在一起，输出下一个 wordpiece 的概率	融合决策

几个术语：

- **端到端 (end-to-end)**：一个网络直接从音频输出文字，不需要单独的发音词典和语言模型。
- **流式 (streaming)**：边说边识别，只能看到当前和过去的音频，适合手机上的实时输入。
- **Conformer**：编码器的具体结构，把 Transformer 的自注意力和卷积结合在一起。
- **Cascading encoders (级联编码器，论文 [18])**：这是背景知识，不是这篇论文展开讲的内容。一般做法是先接一个流式编码器快速出结果，再在后面级联一个能看未来上下文的编码器，输出更准的结果。论文只说生产环境用的是这个变体，没有更多细节。

search. If x are the input acoustic frames, and y_n are the N-best hypotheses for the output label sequence, then the loss can be written as:

$$\mathcal{L}^{\text{MWER}} = \sum_{i=1}^N \hat{P}(y_i|x) \cdot \left(W(y_i, y) - \frac{\sum_{i=1}^N W(y_i, y)}{N} \right) \quad (2)$$

where $\hat{P}$ is the probability of the i -th hypothesis normalized over all N-best hypotheses, and $W(y_i, y)$ is the number of word errors in the i -th hypothesis relative to the ground truth.

MWER loss is a component of server-side training, but would be too computationally expensive for an on-device setting due to the memory cost of computing the N-best hypotheses. Instead, we create a modified version for on-device by caching the hypotheses produced by the model decoder during training, and incorporating them during loss computation.

3.2. Training Data

In all our experiments, the model is pretrained on speech data from a multi-domain dataset (MD), encompassing domains of search, farfield, telephony, YouTube, etc [22, 23], including a Short-Form (SF) and Medium-Form (MF) domain. The model is then fine-tuned on training examples containing the interesting terms. All data are anonymized, and are collected, managed, and used for training models in accordance with Google AI Principles [24].

For our simulation experiments, the fine-tuning dataset consists of utterances from the SF domain that contain long-tail words (SF-LT). Learning is evaluated on the targeted SF-LT testset, a disjoint set of utterances containing the same long-tail words as the training data. Additionally, to ensure the model quality does not degrade on the overall distribution, we also use disjoint testsets from the SF and MF domains to measure regression on non-targeted words.

In production experiments, a similar pre-trained model is fine-tuned on user data through on-device FL, filtered to examples containing the long-tail words. For server-side evaluation, we create a corresponding targeted eval set, as outlined in the methodology. We also make use of the MF testset as our benchmark for preserving quality on the overall distribution. In both these cases, our goal is to improve WER on the targeted testsets, while maintaining WER on the overall distribution testsets.

4. EXPERIMENTAL RESULTS

4.1. In Simulation

As shown in Table 2, we started with a baseline trained on our multidomain dataset. By fine-tuning only on examples containing the long-tail words (SF-LT), we saw a 31% relative improvement over the baseline on the targeted testset, but observed catastrophic forgetting in the form of significant

Dataset	MF WER	SF WER	SF-LT WER
MD (Baseline)	3.7	6.3	45.5
Fine-tuning Whole Model			
SF-LT	7.8	9.3	31.4
SF-LT + SF + MF	4.4	6.2	35.1
Fine-tuning Joint Layer			
SF-LT	6.4	8.4	38.1
SF-LT + SF + MF	3.8	6.3	40.6

Table 2. In simulation, fine-tuning afforded improvement on the targeted testset (SF-LT), but degraded overall testset quality (MF + SF). Reintroducing SF + MF training data mitigated forgetting. When training was limited to the joint layer, the model was still able to improve on the targeted testset, while recovering the original WER for the overall testsets.

degradation on the testsets that represent the overall distribution, including over 100% regression on MF.

As these simulation experiments were done entirely server-side, we were able to address forgetting by reintroducing training examples from the original domain (SF and MF) during training, in equal proportion to the SF-LT dataset, similar to the Mixture of Centralized and Federated Training approach we proposed for production. This way, we were still able to achieve a 23% improvement in the targeted testset, while seeing much less degradation in the overall domain.

In order to understand the most memory-efficient setting, we also tried fine-tuning only the joint layer of the model. Here, we saw less improvement on the targeted testset, but even lower degradation on the overall domain. By fine-tuning on both the targeted data and the overall data distributions, we saw a 11% improvement on the targeted testset, and little or no regression on the overall testsets.

4.2. Necessity of Wordlist Filtering In Production

In production experiments, we began by training on all examples with user edits, but the model quality quickly and dramatically degraded (Figure 3), lending credence to our hypothesis that many of these edits are indeed revisions to the original utterance, rather than corrections of an incorrect ASR transcript, and likely result in significantly divergent text-audio pairs.

However, using our list of fresh words that were likely to be misrecognized, we were able to filter training examples largely to those containing true user corrections. When on-device training was limited to examples with edits that contained a word in our fresh wordlist, the WER was stabilized.

4.3. Catastrophic Forgetting

As shown in Table 3, this approach was able improve model quality on the targeted testset, but we did observe the expected

这个图蛮好的，蓝色的展示的就是完全不带过滤，
没有剔除掉意图改变的训练结果，WER是非常高的
如果没过滤，模型质量很快大幅变差了

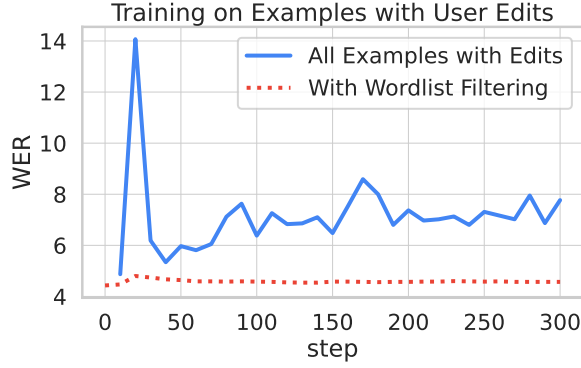


Fig. 3. Training on all examples with user edits caused the model quality to quickly diverge, indicating many were not actual corrections. However, filtering these examples by our wordlist resulted in far more stable model quality.

为什么静态平均能“两全其美”？
这是一个比较反直觉的现象。把
两个模型的参数直接取加权平
均，结果往往不是“两边都一般
般”，而是能同时保留两边的能
力。
前提是两个模型出自同一个起
点，离得不远。论文引用的 [9]
就是研究这类权重平均的工作
在这里，Pure FL 模型是从初始
模型微调出来的，两者很接近，
平均后既保留了新词能力，也恢
复了原来的整体能力。

Setup	Overall WER	Targeted WER
Baseline	4.4	17.5
Pure FL	4.6	16.1
Static Ckpt Avg	4.4	16.1
Dynamic Ckpt Avg	4.4	16.6

Table 3. In a production-like setting, both techniques were able to restore the baseline overall WER, but Static Checkpoint Averaging gave better results on the targeted testset.

forgetting of the overall distribution. From the baseline, we saw that filtering on-device training examples to those containing targeted words allowed us to improve 8% relative on the targeted testset. However, we also saw a 4.5% regression on the overall WER, highlighting the necessity of techniques to mitigate catastrophic forgetting.

4.3.1. Checkpoint Averaging

Both Static Checkpoint Averaging and Dynamic Checkpoint Averaging were able to bring us back to parity on the overall distribution, while still affording improvement on the targeted distribution (Table 3). In particular, with Static Checkpoint Averaging, we were able to achieve the 8% improvement on the targeted testset from pure FL, while keeping the WER on the overall distribution from the baseline.

4.3.2. Mixture of Centralized and Federated Training

To test the technique of mixing centralized training in with the federated rounds, we first refreshed the model with more in-domain training data. This led to a much improved baseline, and a far greater regression after our targeted fine-tuning, as shown in Table 4. This 20% relative gap was greater than weight averaging techniques could address. However, once we mixed in centralized training simultaneously with the fed-

Setup	Overall WER	Targeted WER
Baseline	3.5	16.6
FL-only	4.4	16.1
FL + Centr. Mix	3.5	16.1

Table 4. After refreshing server-side training data, the baseline was much improved (3.5 WER), and regression from forgetting was much greater (20% relative). However, mixing federated and centralized training rounds restored the baseline Overall WER while keeping FL Targeted WER wins.

erated training rounds, we were able to achieve the best of both worlds: keep the wins on Targeted WER from targeted on-device training, while achieving the same best Overall WER from the server-only baseline.

4.4. Word Improvement

4.4.1. Error Correction Percent

To directly understand how much fine-tuning improved model recognition of each wordlist word, we computed an Error Correction Percent according to the following formula:

$$EC\% = \frac{Acc_{exp} - Acc_{base}}{1.0 - Acc_{base}} \quad (3)$$

Each wordlist word appears in multiple testset examples; for each word, the accuracy Acc is the number of examples where the word was correctly recognized, normalized by the total number of examples containing the word. The $EC\%$ shows how many errors made by the baseline model were corrected by the fine-tuned model, as a ratio of the total number of errors for each word. For words with an $EC\%$ of 100%, fine-tuning corrected all errors made by the baseline.

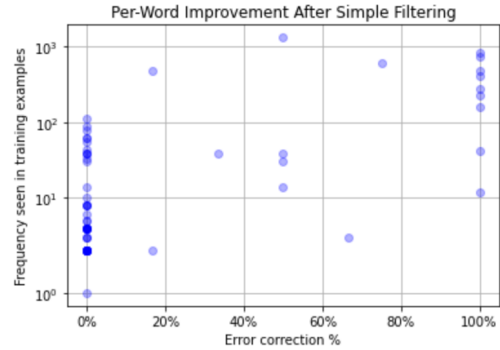


Fig. 4. Comparing number of errors before and after on-device fine-tuning, most words seen at least 100 times had all errors corrected after FT. This suggests that more training examples are needed for the remainder.

Looking closely at how each wordlist word was improved in Figure 4, we saw that the majority of words that were seen

Setup	Overall WER	Tgt WER
Baseline	4.4	17.5
Simple Fine-Tuning	4.6	16.1
Static Ckpt Avg	4.4	16.1
Client Loss Weighting	4.4	16.0
Probabilistic Sampling	4.6	15.8
Prob Samp + St Ckpt Avg	4.5	15.9

Table 5. Client Loss Weighting was able to slightly improve our already fine-tuned targeted WER from 16.1 to 16.0. Probabilistic Sampling did even better, improving our targeted WER to 15.8. The WER on the overall distribution can be restored by reintroducing checkpoint averaging.

at least 100 times in training had 100% of their errors fixed by the experiment. Many words that were seen less frequently were also improved, but errors were not fixed in all their occurrences. This suggested that it was important to increase the frequency of training examples of words that we rarely saw, motivating our probabilistic sampling approach.

4.4.2. Probabilistic Sampling and Client Loss Weighting

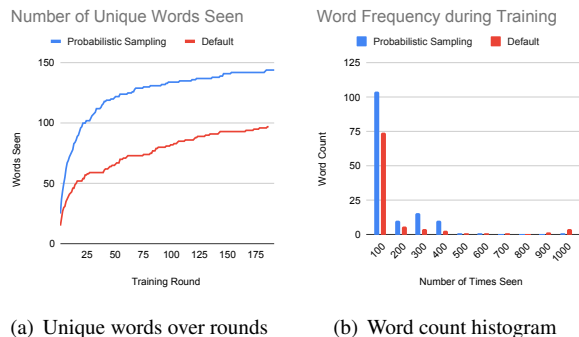


Fig. 5. Probabilistic sampling was able to improve the number of unique words seen, as well as the number of words that were seen at least 100 times.

By adding Probabilistic Sampling during fine-tuning, we increased the number of unique words seen from around 100 to closer to 150. Additionally, we increased the number of words seen at least 100 times (Fig. 5).

As shown in Table 5, in comparison with our previous best fine-tuned result of 16.1, using Client Loss Weighting was able to give us a small improvement to 16.0. Probabilistic Sampling gave more significant wins, bringing our targeted WER down to 15.8.

4.4.3. Contact Names

As an additional application, we turned these techniques towards a related domain. Rather than learning fresh terms,

Setup	Overall WER	Contact Names WER
Baseline	3.8	5.7
Fresh Terms Filter	3.8	5.5
Names Filter	3.8	5.4

Table 6. Contact name recognition was improved by the above techniques. Using a names wordlist for filtering instead gave even better results, without harming the Overall WER.

we experimented with whether we could improve recognition of another class of commonly misrecognized terms: names from users’ contacts lists. Names are difficult to transcribe correctly due to variety and alternate spellings, and benefit greatly from a diverse and fresh training corpus.

Even using just the fresh wordlist filtering, training in FL was sufficient to improve the WER on our Contact Names testset from 5.7 to 5.5, as shown in Table 6. Though we did not target any names with our filtering, the general exposure to fresh training examples was able to improve the WER. We further improved this by replacing the wordlist with a list of the top 1000 baby names in each the US, China, and India. With this new setting, we reached 5.4 WER on the testset, a 5% relative improvement.

This demonstrates that the proposed techniques can be applied to other settings, such as learning words of a certain domain, while maintaining quality on the overall distribution.

5. CONCLUSION

In this work, we explored how user corrections can be leveraged to improve ASR model quality, particularly on fresh terms not available during model training, or on terms that the ASR model tends to get wrong. The naive approach of training directly on novel on-device user data sources posed a number of difficulties, including the challenge of targeting these inherently long-tail words, as well as catastrophic forgetting leading to degradation of model quality on the overall distribution.

To address these issues, we applied a number of techniques, such as checkpoint averaging, mixing centralized and decentralized training, and probabilistic sampling. **Finally, we experimentally demonstrated the potential of this technique, both for the original problem space of learning fresh terms, as well as in adapting the model to other difficult domains, such as names. We hope that these findings may lead to further exploration of adapting ASR models to an ever-changing language corpus.**

6. ACKNOWLEDGEMENTS

We thank Yonghui Xiao, Andrew Hard, Sean Augenstein, Khe Chai Sim, Guru Prakash, and Tien-Ju Yang for their valuable contributions to this work.

7. REFERENCES

- [1] K. Bonawitz, H. Eichner, W. Grieskamp *et al.*, “Towards federated learning at scale: System design,” 2019.
- [2] H. B. McMahan, E. Moore *et al.*, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA*, ser. Proceedings of Machine Learning Research, A. Singh and X. J. Zhu, Eds., vol. 54. PMLR, 2017, pp. 1273–1282. <http://proceedings.mlr.press/v54/mcmahan17a.html>
- [3] J. Xue, J. Han, T. Zheng *et al.*, “Hard sample mining for the improved retraining of automatic speech recognition,” 2019.
- [4] L. Qu, C. Weber, and S. Wermter, “Emphasizing unseen words: New vocabulary acquisition for end-to-end speech recognition,” *Neural Networks*, vol. 161, pp. 494–504, apr 2023. <https://doi.org/10.1016%2Fj.neunet.2023.01.027>
- [5] K. C. Sim, F. Beaufays, A. Benard *et al.*, “Personalization of end-to-end speech recognition on mobile devices for named entities,” 2019.
- [6] U. Alon, G. Pundak, and T. N. Sainath, “Contextual speech recognition with difficult negative training examples,” 2018.
- [7] R. M. French, “Catastrophic forgetting in connectionist networks,” *Trends in Cognitive Sciences*, vol. 3, no. 4, pp. 128–135, 1999.
- [8] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” ser. Psychology of Learning and Motivation, G. H. Bower, Ed. Academic Press, 1989, vol. 24, pp. 109–165.
- [9] S. V. Eeckht and H. V. hamme, “Weight averaging: A simple yet effective method to overcome catastrophic forgetting in automatic speech recognition,” 2023.
- [10] S. Augenstein, A. Hard, L. Ning *et al.*, “Mixed federated learning: Joint decentralized and centralized learning,” 2022.
- [11] W. Zhu, P. Kairouz, B. McMahan *et al.*, “Federated heavy hitters discovery with differential privacy,” 2020.
- [12] A. van den Oord, Y. Li, I. Babuschkin *et al.*, “Parallel wavenet: Fast high-fidelity speech synthesis,” 2017.
- [13] A. Graves, A. Mohamed, and G. Hinton, “Speech Recognition with Deep Recurrent Neural Networks,” in *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013, pp. 6645–6649.
- [14] T. N. Sainath, Y. He, B. Li *et al.*, “A streaming on-device end-to-end model surpassing server-side conventional model quality and latency,” 2020.
- [15] Y. He, T. N. Sainath, R. Prabhavalkar *et al.*, “Streaming end-to-end speech recognition for mobile devices,” 2018.
- [16] A. Gulati, C.-C. Chiu, J. Qin *et al.*, Eds., *Conformer: Convolution-augmented Transformer for Speech Recognition*, 2020.
- [17] B. Li, A. Gulati, J. Yu *et al.*, “A better and faster end-to-end model for streaming asr,” 2021.
- [18] T. N. Sainath, Y. He, A. Narayanan *et al.*, “An efficient streaming non-recurrent on-device end-to-end model with improvements to rare-word modeling,” in *Proc. of INTERSPEECH*, 2021.
- [19] Z. Allen-Zhu, Y. Li, and Y. Liang, “Learning and generalization in overparameterized neural networks, going beyond two layers,” *CoRR*, vol. abs/1811.04918, 2018. <http://arxiv.org/abs/1811.04918>
- [20] B. Neyshabur, Z. Li, S. Bhojanapalli *et al.*, “Towards understanding the role of over-parametrization in generalization of neural networks,” *CoRR*, vol. abs/1805.12076, 2018. <http://arxiv.org/abs/1805.12076>
- [21] R. Prabhavalkar, T. N. Sainath, Y. Wu *et al.*, “Minimum word error rate training for attention-based sequence-to-sequence models,” 2017.
- [22] A. Misra, D. Hwang, Z. Huo *et al.*, “A Comparison of Supervised and Unsupervised Pre-Training of End-to-End Models,” in *Proc. Interspeech 2021*, 2021, pp. 731–735.
- [23] A. Narayanan, R. Prabhavalkar, C.-C. Chiu *et al.*, “Recognizing long-form speech using streaming end-to-end models,” in *IEEE Automatic Speech Recognition and Understanding Workshop, ASRU 2019, Singapore, December 14-18, 2019*. IEEE, 2019, pp. 920–927. <https://doi.org/10.1109/ASRU46091.2019.9003913>
- [24] Google, “Artificial intelligence at Google: Our principles.” <https://ai.google/principles/>